PROJECT REPORT

Course: Computer Science and Engineering (Artificial Intelligence and Machine Learning)

Submitted by: Utkarsh…Registration Number = 25BAI11231

Submitted to: Dr. Vinesh Kumar

Date: 30 March 2026

Cover Page

Project Title: Smart Weather and Crops Recommendation Analyser

Project Goal: To develop an intelligent system that combines Machine Learning techniques with simulated weather parameters to generate and assist accurate crop recommendations for cultivation in a given state and season.

Author: Utkarsh

Institution: VIT Bhopal University

Date of Submission: 30 March 2026

Project Introduction

Agriculture is a critical sector where productivity relies heavily on environmental conditions such as temperature, humidity and rainfall. The agricultural sector is the backbone of the global economy, yet it remains one of the most volatile industries due to its heavy dependence on environmental factors.

With the advancement of Artificial Intelligence and Machine Learning, it is now possible to build intelligent systems that assist farmers in making informed decisions.

This project "**Smart Weather and Crop Recommendation Analyzer"** is an intelligent system designed to predict the most suitable crop for a specific plot of land, depending upon climatic inputs. It integrates a machine learning model with a graphical user interface (GUI), allowing users to input environmental data and receive crop recommendations instantly.

This project demonstrates how **data-driven approaches** can replace traditional guess-based farming decisions and improve agricultural efficiency. The system minimizes the risk of crop failure and optimizes yield. This project leverages Python's robust data science ecosystem to bridge the gap between complex meteorological data and actionable farming insights.

Problem Statement

Many beginner-level Python learners struggle to understand GUI development and how data can be collected and stored persistently.

The primary problem this project addresses is the need for a **digital, standardized, and secure pipeline** to acquire, validate, and store data related to weather and climatic conditions such as temperature, wind speed, rainfall, humidity etc. and seasonal crops of a state.

Farmers often face uncertainty in choosing the right crop due to changing weather patterns and lack of access to analytical tools. Farmers often face "Information Asymmetry," where they lack scientific data about their soil's health or upcoming weather shifts. It leads to:

- **Improper Crop Selection:** Planting crops that are incompatible with the soil.
- **Resource Inefficiency:** Over-application of fertilizers due to a lack of understanding of existing soil chemistry.
- **Climate Vulnerability:** Ignoring humidity and rainfall patterns, which are critical for crop maturity.
- **Economic Loss:** High input costs with low harvest output due to trial-and-error farming.

Incorrect crop selection leads to reduced yield, financial loss, and inefficient resource utilization.

There is a need for a system that:

- Analyses environmental factors
- Provides accurate crop recommendations
- Is accessible to non-technical users.

Functional Requirements

The system must perform the following actions:

1. User should be able to input temperature.
2. User should be able to input humidity.
3. User should be able to input rainfall.
4. User should select state/region.
5. System should process inputs using ML model.
6. System should display recommended crop.

- GUI should allow easy interaction.
- The system must reject negative values or unrealistic soil parameters.

Non-Functional Requirements

These define the quality attributes and constraints of the system.

- The GUI must be intuitive, responsive, and easy to navigate for non-technical users.
- The recommendation engine should maintain a predictive accuracy of over 90% based on the training dataset.
- **Performance:** The system should return a recommendation within 2–3 seconds of data submission.
- **Scalability:** The architecture should allow for the addition of more crop types or soil parameters in the future without a complete rewrite.
- **Portability:** The system should run on any system with Python on any system with Python.
- **Maintainability:** The system must have Modular code structure.

System Architecture

The system follows a simple layered architecture:

- Presentation Layer: Tkinter GUI
- Application Layer: Input processing logic
- ML Layer: Decision Tree Model
- Data Layer: Custom-build Pandas DataFrame

Architecture Flow

The data flows in a linear sequence to ensure integrity:

- User inputs environmental data via GUI.
- Input is pre-processed. The combined vector of all the attributes (Soil + Weather) is passed to the **Preprocessor**.
- Data is passed to trained ML model
- Model predicts suitable crop.
- Result is displayed to user. The model outputs a Label (e.g., "Rice"), which is mapped to a human-readable string and displayed to the user.

- User Input → Validation → Analysation → Recommendation/Errors

Design Diagrams

- **Use Case:**
  Actor: User
  Actions: Input Soi l Data, Request Weather Update and View Recommendation.

- Enter Environmental Data
- Select Region
- Get Crop Recommendation

  Fill Form → Validate Data →Analyse Data → Show Recommendation.
- **Workflow diagram:** User → Input data→ Data Preprocessing→ ML

  Model Prediction→ Display output (Recommendation).

- **Sequence Flowchart:**
  Start
  Input Parameters
  Validate Input
  Load Trained Model
  Process Data
  Predict Crop
  Display Result
  End

Design Decisions and Rationale

Design Decision	Rationale
Decision Tree Model	Easy to interpret and fast
Tkinter GUI	Chosen as it is the standard, simple lightweight, and pre-installed GUI library for Python, minimizing external dependencies.
Programming Language: Python	Chosen for its extensive library support and excellent readability and strong ML support, making the project maintainable and scalable.
Static Dataset	Simplifies development
Modular Design	Easy maintenance and scalability

Moreover, the Decision Tree was selected because it provides **clear decision boundaries** and is suitable for classification tasks with structured data.

Implementation Details

- **Programming Language and GUI used:** Python 3.13 64-bit, Tkinter for GUI.
- **ML Model:** Decision Tree
- **Data Handling:** Pandas

- **Randomisation:** Random Library

- **Key Modules:**
- Input Handling Module
- Data Preprocessing Module
- Model Training Module
- Prediction Module
- GUI Module

Program Execution and Results

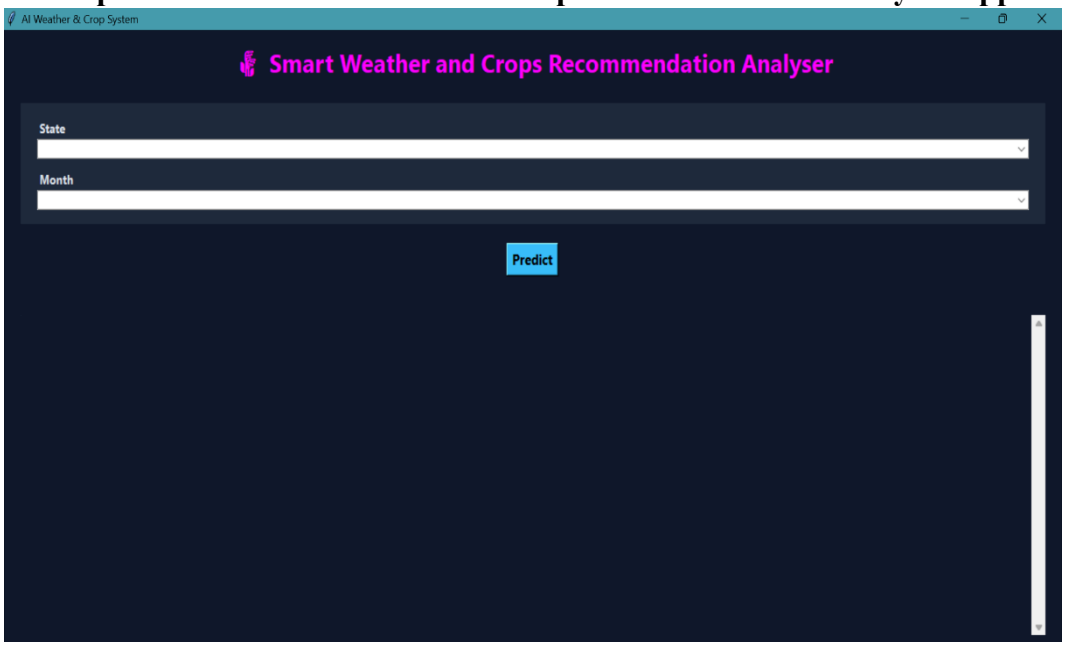
- The project successfully meets all functional requirements.
- A user is able to interact with the system and all submitted data is validated in real-time.

  Upon successful submission, an output containing the analysed is correctly displayed at the Front-End. Upon execution, the user is greeted with a sidebar for inputs.

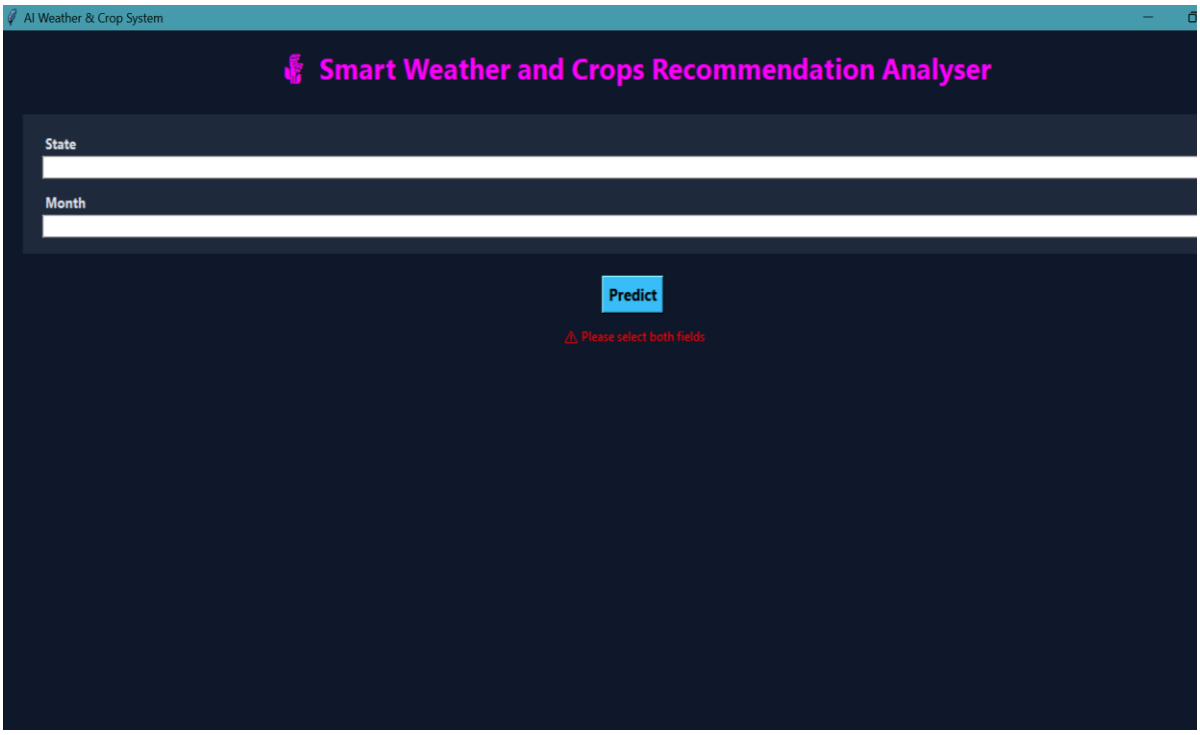
- Execution steps:
  - Run Python script
  - GUI window opens
  - Enter inputs
  - Click "Predict"
  - Output displayed

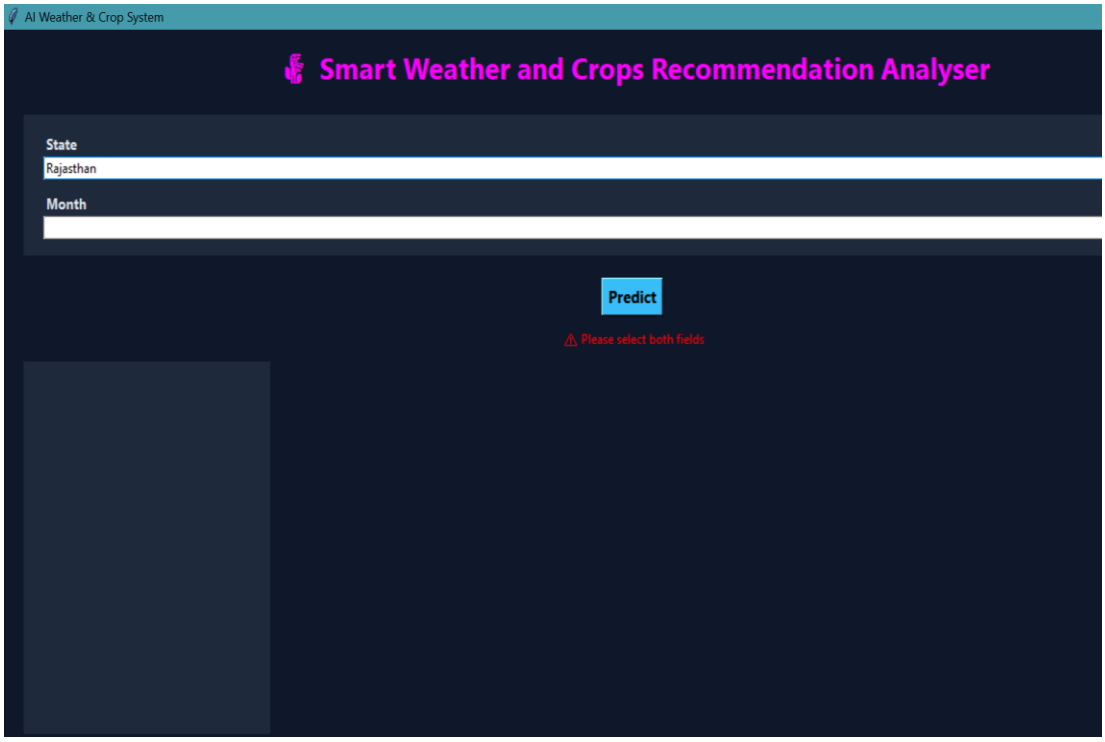
Here, I am pasting my all the results of my programs when executed:

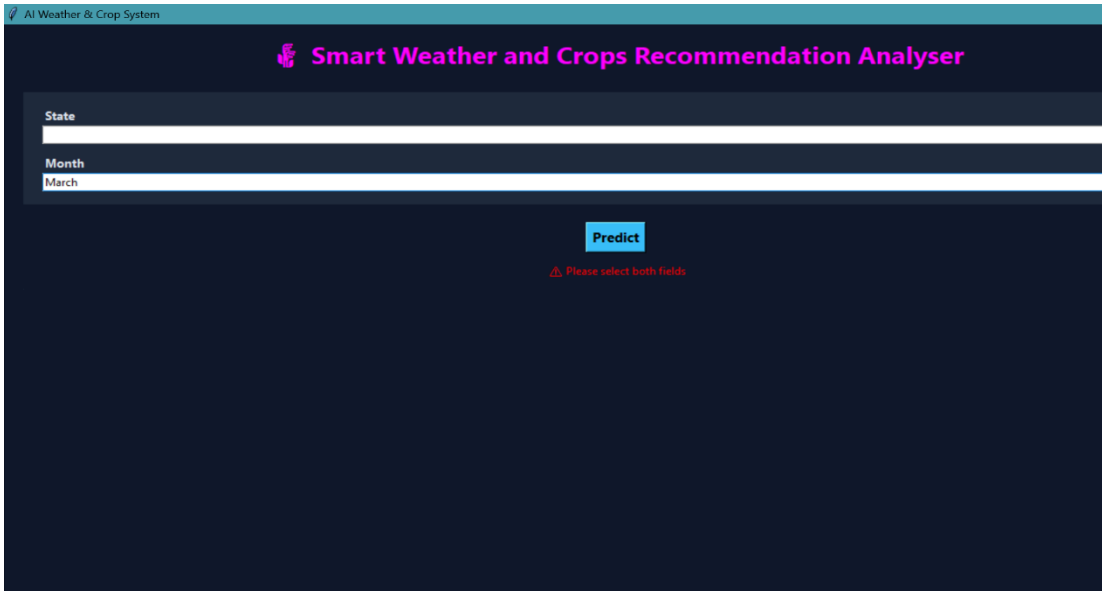
1. **Development of Smart Weather and Crop Recommendation Analyser App**



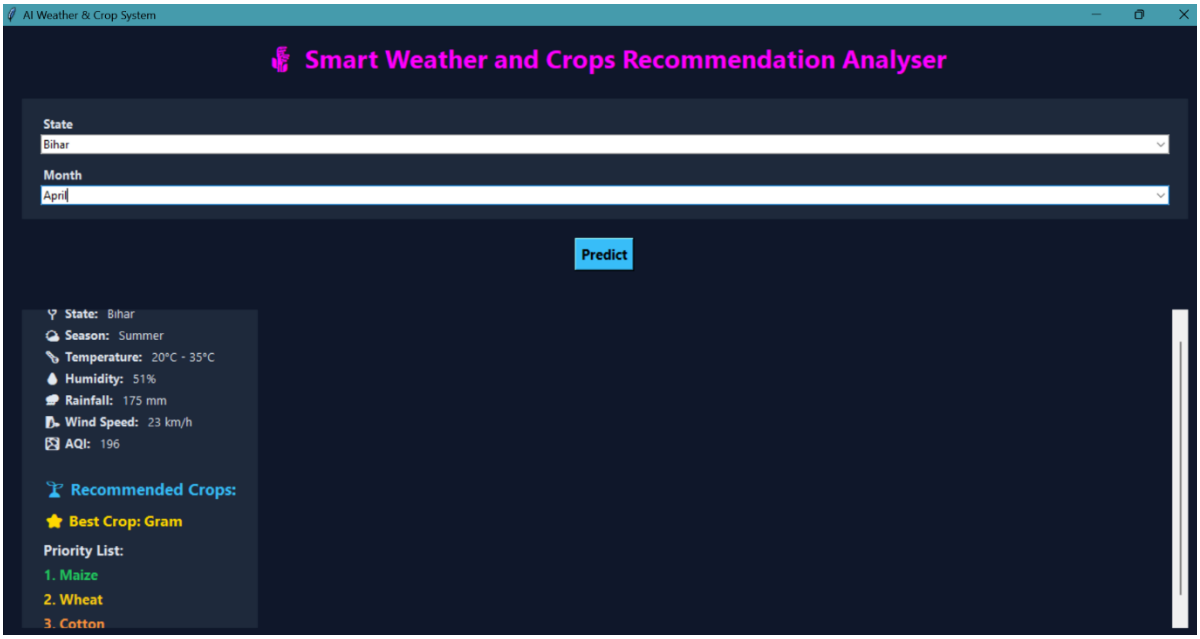
2. **Input Status: Unfilled Fields**







3. Successful Execution



Observations:

- System provides instant predictions.
- Results are consistent with dataset patterns.
- GUI interaction is smooth.

Testing Approach

This project performs multi-stage testing approach:

- **Unit Testing:** Individual modules are tested.
- **Integrated Testing:** GUI + Model are tested together.
- **Functional Testing:** Verified input-output correctness.
- **User Testing:** Checked usability.
- **Edge Case Testing:** Extreme values are tested.

Challenges Faced

- We have to maintain input validation for multiple fields, consecutively and we are also required to handle invalid user inputs.
- We have to handle file operations correctly.
- We have to keep our codes well-structured.
- Our all the codes must be modular.
- There is a limited availability of real-world datasets.
- Data preprocessing and normalization is also a challenge.
- Integrating ML model with GUI is sometimes a challenging task.
- We do always need to ensure prediction accuracy.

Learnings and Key Takeaways

These types of simple projects for beginners provides them valuable experience in several key areas of application development:

- Gain in hands-on experience with Python Tkinter.
- Improved understanding of file handling and data storage.
- Enhanced knowledge of GUI event-driven programming.
- Experience about how separation of GUI, logic and data dramatically simplifies debugging and maintenance.
- **Full-Stack Data Science:** The project provided experience in not just building a model, but creating a functional end-to-end product.
- **Data Cleaning:** Realized that a major portion (more than 60%) of a successful project lies in data cleaning and preprocessing rather than just the algorithm choice.
- Experience in model training and prediction**.**

Limitations

- It uses a static dataset (no real-time data).
- Limited environmental parameters are featured.
- Does not include soil analysis.
- Accuracy depends on dataset quality
- Desktop-only application (no web/mobile version featured).

Potential Future Enhancements

To expand and enhance the functionality and professionalize the application, the following features could be added:

- **Modern GUI:** Migrate the application from "Tkinter" to a more modern library like "Flet", "PyQt", BeeWare", etc. for enhanced aesthetics and complex layout management.
- Integration with real-time weather **APIs** like OpenWeatherMap API.
- We can add **soil analysis** features. This includes the use of Soil Data (elements like N, P, K and pH) via the web interface.
- **IoT Integration:** Connecting physical soil sensors to the Python app for real-time automated data entry.
- Using advanced models like:
    - ✓ Random Forest
    - ✓ Gradient Boosting
    - ✓ Neural Networks

- **Market Integration:** Adding a feature that suggests crops based on current market demand/price.
- **Web/mobile application** deployment: It can be done by converting the web dashboard into a lightweight Android application for offline use.
- These features can too be added:
  Multilingual support
  Cloud-based system.
- **Fertilizer Recommendation:** Expanding the model to suggest not just the crop, but the specific amount of fertilizer needed to fix soil deficiencies.

References

- Sci-kit learn documentation.
- GeeksforGeeks Python GUI Tutorials.
- YouTube lectures to develop basic programming skills in Python Programming Language (College Wallah).
- VITyarthi
- AI tools ChatGPT.